Jostar: A Feature Selection Library for Data Science in Python

Amir Hassanzadeh and Jan van Aardt

Chester F. Carlson Center for Imaging Science, Rochester Institute of Technology, Rochester, NY 14623 USA

INTRODUCTION

Feature selection approaches are widely used in all areas of data science, from remotely sensed data for agriculture practices, to applications in medical field such as for cancer prediction[1]. However, the Python community lacks a comprehensive and user-friendly module for feature selection. Jostar, derived from the Farsi word جستار meaning finder, is a Python-based feature selection module comprised of 9 different feature selection algorithms from single objective to multi-objective methods, dedicated to solve regression and classification problems. The algorithms, to this date, are:

1) Ant Colony Optimization (ACO)
2) Differential Evolution (DE)
3) Genetic Algorithm (GA)
4) Plus-L Minus-R (LRS)
5) Non-dominated Sorting Genetic Algorithm (NSGAII)
6) Particle Swarm Optimization (PSO)
7) Simulated Annealing (SA)
8) Sequential Forward Selection (SFS)

FEATURES

Jostar comes with multiple features, three major features are listed below. Beginning with loading libraries and required models we have:

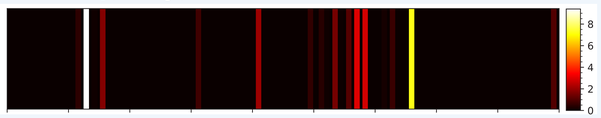
```
from jostar.algorithms import ACO; import numpy as np
from sklearn.svm import SVC; from sklearn.metrics import f1_score
x, y  = LoadData()
model = SVC()
```

1. Tune hyperparameters with the help of Hyperopt[2]:

```
from hyperopt import hp; import numpy as np
from jostar.utils import ParamTuner
param_dict = {
      'alpha': hp.loguniform('alpha', np.log(1e-5), np.log(0.5)),
      'rho': hp.loguniform('rho', np.log(1e-5), np.log(0.5)),
      'tau0': hp.loguniform('tau0', np.log(1e-5), np.log(1))}
aco = ACO(model, n_f = 5, weight = 1, scoring = f1_score)
pt = ParamTuner(aco, x, y)
best_params = pt.tune(param_dict, max_evals=200)
```

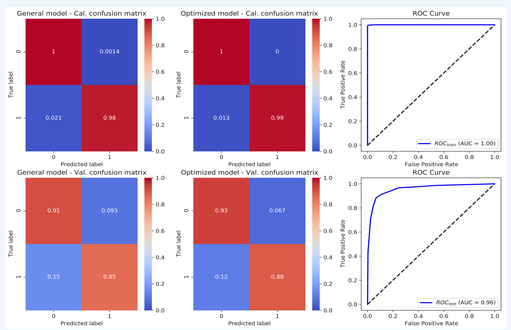
2. Generate feature rankings with our proposed ranking method:

```
aco = ACO(model, n_f = 5, weight = 1, scoring = f1_score)
rankings = aco.rankings
PlotRanking(rankings)
```



3. Display classification and regression results:

```
aco = ACO(model, n_f = 5, weight = 1, scoring = f1_score)
aco.display_results()
```



Jostar

Identify Discriminating Features with just a few lines of code, similar to Sklearn.

```
from sklearn.cross_decomposition import PLSRegression
from jostar.algorithms import ACO
from sklearn.metrics import r2_score

# Load data
x, y  = LoadData()

# Load Sklearn model
model = PLSRegression()

# Load Jostar optimization model
aco = ACO(model=model, n_f=5, weight=1, scoring=r2_score)

# Call fit
aco.fit(x,y)
```

DOCUMENTATION

There is an overall uncertainty regarding parameters of heuristic algorithms. Our literature review helped us create a powerful inline documentation. Below is the in-line documentation for Simulated Annealing, as an example:

```
class SA(BaseFeatureSelector):
    def __init__(self, model, n_f, weight, scoring, n_iter= 1000, n_sub_iter=100,
                 cv=None, t0 = 1e-1, alpha= 0.9, verbose= True,
                 random_state=None,**kwargs):
        """
        Simulated Annealing (SA) was introduces by PJM. Laarhoven and EHL Aarts
        1987. SA is usually used in discrete spaces and mimics the
        physical annealing. SA is widely used in Travelling Salesman problem.

        Parameters
        ----------
        model : class
            Instantiated Sklearn regression or classification estimator.
        n_f : int
            Number of features needed to be extracted.
        weight : int
            Maximization or minimization objective, for maximization use +1
            and for mimization use -1.
        scoring : callable
            Callable sklearn score or customized score function that takes in y_pred
and y_true
        n_iter : int, optional
            Maximum number of iterations. For more complex
            problems it is better to set this parameter larger.
            The default is 1000.
        n_sub_iter : int, optional
         Maximum number of sub iterations (number of neighbourhood solutions)
            per iteration. For more complex problems it is better to set this
            parameter larger.
            The default is 100.
        cv : class, optional
            Instantiated sklearn cross-validation class. The default is None.
        t0 : float, optional
            The initial temperature parameter. There is no reported default value
            for this parameter and determining the initial temperateure is an
            advance topic. It is suggested that users choose a value between
            [0.1,500] and tune it. The defualt is 0.1.
        alpha : float, optional
            Cooling factor/ratio parameter. The value can reside [0,1] but
            usually a value of [0.8,0.99] fits the best. The default is 0.9.
        verbose : bool, optional
            Wether to print out progress messages. The default is True.
        random_state : int, optional
            Determines the random state for random number generation,
            cross-validation, and classifier/regression model setting.
            The default is None.

        Returns
        -------
        Instantiated Optimziation model class.

        References:
            Park, M. W., & Kim, Y. D. (1998). A systematic procedure for
            setting parameters in simulated annealing algorithms. Computers &
            Operations Research, 25(3), 207-217.
```

Github

INSTALLATION

Installing Jostar is easy. Use pip as below to install jostar.

```
$ pip install jostar
```

ACKNOWLEDGEMENT

Jostar is the Python implementation and expansion of YPEA[3], a MATLAB toolbox for solving optimization problems using evolutionary algorithms and meta-heuristics.

REFERENCES

[1] Dua, D. and Graff, C. (2019). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.
[2] Bergstra, J., Yamins, D., & Cox, D. D. (2013, June). Hyperopt: A python library for optimizing the hyperparameters of machine learning algorithms. In Proceedings of the 12th Python in science conference (Vol. 13, p. 20).
[3] Mostapha Kalami Heris, Yarpiz Evolutionary Algorithms Toolbox for MATLAB (YPEA), Yarpiz, 2020.